KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
https://jti.polinema.ac.id

Mata Kuliah : Pemrograman Mobile
Program Studi : D4 – Teknik Informatika
Semester : 5

Kelas : 2A TI
NIM : 244107020112
Nama : Muhammad Shabran
Pertemuan : 6

LAPORAN PRAKTIKUM

Praktikum 1: Login + secure storage + token refresh

Langkah	Keterangan
1.	Persiapan Project

```
PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course> flutter create week6_campus_notify
Creating project week6_campus_notify...
Resolving dependencies in `week6_campus_notify`... (2.2s)
Downloading packages...
Got dependencies in `week6_campus_notify`.
Wrote 131 files.

All done!
You can find general documentation for Flutter at: https://docs.flutter.dev/
Detailed API documentation is available at: https://api.flutter.dev/
If you prefer video documentation, consider: https://www.youtube.com/c/flutterdev

In order to run your application, type:

  $ cd week6_campus_notify
  $ flutter run

Your application code is in week6_campus_notify\lib\main.dart.
```

```
PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course> cd week6_campus_notify
PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course\week6_campus_notify> flutter pub add flutter_
riverpod go_router dio flutter_secure_storage
Resolving dependencies...
Downloading packages... (3.0s)
+ args 2.7.0
+ code_assets 1.2.1 (2.1.0 available)
+ crypto 3.0.7
  cupertino_icons 1.0.9 (2.0.0 available)
+ cupertino_ui 1.1.2
+ dio 5.11.1
+ dio_web_adapter 2.2.2
+ ffi 2.2.0
+ ffi_leak_tracker 0.1.2
+ fixnum 1.1.1
+ flutter_localizations 0.0.0 from sdk flutter
+ flutter_riverpod 3.4.3
+ flutter_secure_storage 11.2.0
+ flutter_secure_storage_darwin 0.4.3
+ flutter_secure_storage_linux 3.0.3
+ flutter_secure_storage_platform_interface 2.1.1
+ flutter_secure_storage_web 2.1.1
+ flutter_secure_storage_windows 4.2.2
+ flutter_web_plugins 0.0.0 from sdk flutter
+ go_router 18.0.2
+ hooks 2.0.2 (2.2.0 available)
+ http_parser 4.1.2
+ intl 0.20.3
+ jni 1.1.0
+ jni_flutter 1.0.4+1
+ jni_util 1.0.0
+ listen 1.0.1
+ logging 1.3.0
  material_color_utilities 0.13.0 (0.13.1 available)
+ material_ui 1.6.0
```

```
PS C:\Users\muham\Documents\Tugas Kuliah\Semester 5\Mobile\244107020112_mobile_course\week6_campus_notify> flutter pub add firebase
_core firebase_messaging flutter_local_notifications
Resolving dependencies...
Downloading packages... (6.1s)
+ _flutterfire_internals 1.3.77
  code_assets 1.2.1 (2.1.0 available)
  cupertino_icons 1.0.9 (2.0.0 available)
+ dbus 0.7.15 (0.8.0 available)
+ firebase_core 4.15.0
+ firebase_core_platform_interface 8.1.1
+ firebase_core_web 3.12.0
+ firebase_messaging 16.7.0
+ firebase_messaging_platform_interface 4.10.0
+ firebase_messaging_web 4.2.5
+ flutter_local_notifications 22.3.1
+ flutter_local_notifications_linux 8.0.1
+ flutter_local_notifications_platform_interface 12.2.0
+ flutter_local_notifications_web 1.0.0
+ flutter_local_notifications_windows 3.1.1
  hooks 2.0.2 (2.2.0 available)
+ http 1.6.0
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
  objective_c 9.5.0 (9.6.2 available)
+ petitparser 7.0.2 (7.1.0 available)
  record_use 0.6.0 (1.1.1 available)
  test_api 0.7.12 (0.7.15 available)
+ timezone 0.11.1
  vector_math 2.4.0 (2.4.3 available)
+ xml 7.0.1 (7.1.0 available)
Changed 17 dependencies!
12 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
```

2. Penyimpanan token yang aman

```dart
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

class TokenStore {
  TokenStore({FlutterSecureStorage? storage})
      : _storage = storage ?? const FlutterSecureStorage();

  final FlutterSecureStorage _storage;
  static const _accessKey = 'access_token';
  static const _refreshKey = 'refresh_token';

  Future<void> save({required String access, required String refresh}) async {
    await _storage.write(key: _accessKey, value: access);
    await _storage.write(key: _refreshKey, value: refresh);
  }

  Future<String?> readAccess() => _storage.read(key: _accessKey);
  Future<String?> readRefresh() => _storage.read(key: _refreshKey);

  Future<void> clear() => _storage.deleteAll();
}
```

3. Repository auth (mock yang siap diganti Firebase Auth)

```dart
class AuthSession {
  const AuthSession({required this.access, required this.refresh});
  final String access;
  final String refresh;
}

class AuthRepository {
  // GANTI titik ini dengan FirebaseAuth.instance.signInWithEmailAndPassword
  // atau GoogleSignIn saat backend Firebase sudah siap.
  Future<AuthSession> login(
      {required String email, required String password}) async {
    await Future.delayed(const Duration(milliseconds: 500));
    if (!email.contains('@') || password.length < 6) {
      throw Exception('Email atau kata sandi tidak valid');
    }
    // Simulasi JWT: header.payload.signature (jangan parse manual di produksi,
    // gunakan verifikasi server).
    return AuthSession(
      access: 'mock-access-for-$email',
      refresh: 'mock-refresh-for-$email',
    );
  }

  Future<String> refresh(String refreshToken) async {
    await Future.delayed(const Duration(milliseconds: 300));
    if (refreshToken.isEmpty) throw Exception('Refresh token hilang');
    return 'mock-access-renewed-${DateTime.now().millisecondsSinceEpoch}';
  }
}
```

4. Dio dengan refresh otomatis

```dart
import 'package:dio/dio.dart';
import 'auth_repository.dart';
import 'token_store.dart';

Dio buildApiClient(TokenStore store, AuthRepository auth) {
  final dio = Dio(BaseOptions(baseUrl: 'https://example-campus-api.test'));
  dio.interceptors.add(InterceptorsWrapper(
    onRequest: (options, handler) async {
      final access = await store.readAccess();
      if (access != null) {
        options.headers['Authorization'] = 'Bearer $access';
      }
      handler.next(options);
    },
    onError: (e, handler) async {
      if (e.response?.statusCode == 401) {
        final refresh = await store.readRefresh();
        if (refresh == null) return handler.next(e);
        try {
          final renewed = await auth.refresh(refresh);
          await store.save(access: renewed, refresh: refresh);
          final retry = await dio.fetch(
            e.requestOptions..headers['Authorization'] = 'Bearer $renewed',
          );
          return handler.resolve(retry);
        } catch (_) {
          await store.clear(); // refresh ikut mati -> paksa login ulang
        }
      }
      handler.next(e);
    },
  ));
  return dio;
}
```

5. Provider auth + guard route

Providers:

```dart
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../data/token_store.dart';
import '../data/auth_repository.dart';

// Provider Pendukung
final tokenStoreProvider = Provider((ref) => TokenStore());
final authRepositoryProvider = Provider((ref) => AuthRepository());

// Provider Utama
final authStateProvider =
    AsyncNotifierProvider<AuthNotifier, bool>(AuthNotifier.new);

class AuthNotifier extends AsyncNotifier<bool> {
  @override
  Future<bool> build() async {
    final token = await ref.watch(tokenStoreProvider).readAccess();
    return token != null;
  }

  Future<void> login(String email, String password) async {
    state = const AsyncLoading();
    state = await AsyncValue.guard(() async {
      final session = await ref
          .read(authRepositoryProvider)
          .login(email: email, password: password);
      await ref
          .read(tokenStoreProvider)
          .save(access: session.access, refresh: session.refresh);
      return true;
    });
  }

  Future<void> logout() async {
    await ref.read(tokenStoreProvider).clear();
    ref.invalidateSelf();
  }
}
```

Go router:

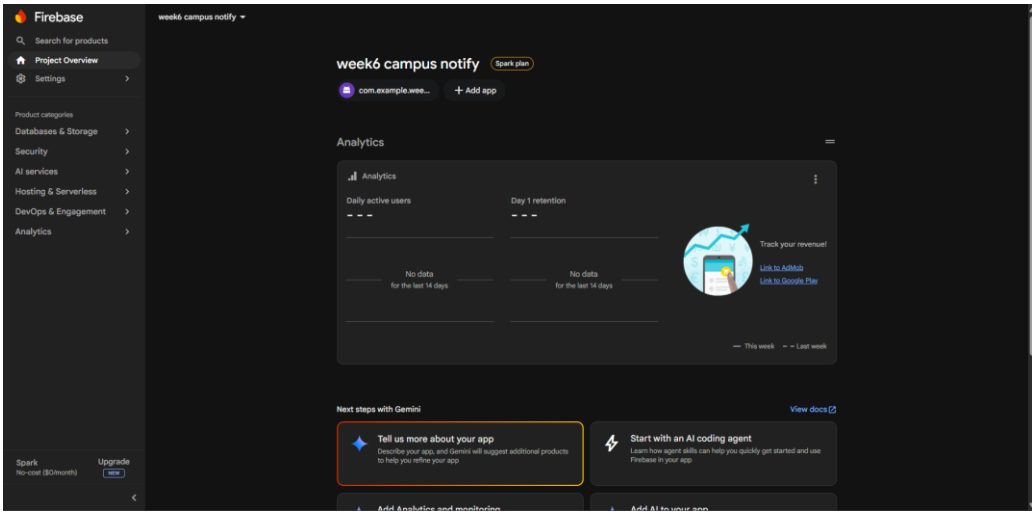
```dart
import 'pages/home_page.dart';
import 'pages/announcement_page.dart';

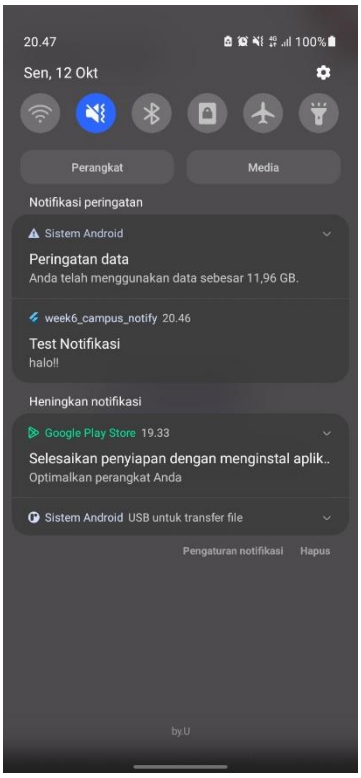
void main() {
  runApp(const ProviderScope(child: MyApp()));
}

final routerProvider = Provider<GoRouter>((ref) {
  return GoRouter(
    initialLocation: '/login',
    redirect: (context, state) {
      final loggedIn = ref.read(authStateProvider).value ?? false;
      final goingLogin = state.matchedLocation == '/login';

      if (!loggedIn && !goingLogin) return '/login';
      if (loggedIn && goingLogin) return '/';
      return null;
    },
    routes: [
      GoRoute(
        path: '/login',
        builder: (context, state) => const LoginPage(),
      ),
      GoRoute(
        path: '/',
        builder: (context, state) => const HomePage(title: 'Flutter Demo Home Page'),
      ),
      GoRoute(
        path: '/pengumuman/:id',
        builder: (context, state) => AnnouncementPage(
          id: state.pathParameters['id'] ?? '',
        ),
      ),
    ],
  );
});
```

Praktikum 2: FCM, permission, dan token lifecycle

Langkah	Keterangan
1.	Daftarkan Aplikasi ke Firebase 
2.	Minta izin notifikasi  Tampilan Token dihalaman debug :

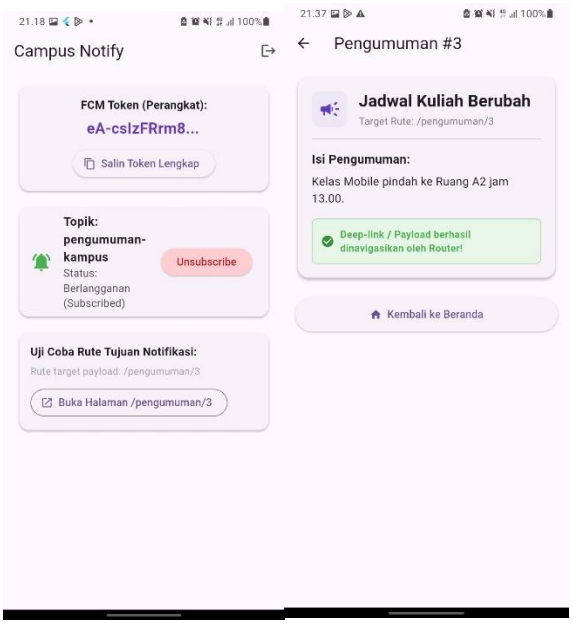
<table>
<tr><td></td><td> 
</td></tr>
<tr><td>3.</td><td>Uji kirim pertama dari Firebase Console
 
</td></tr>
</table>

Praktikum 3: Payload, tiga app state, klik dan topik

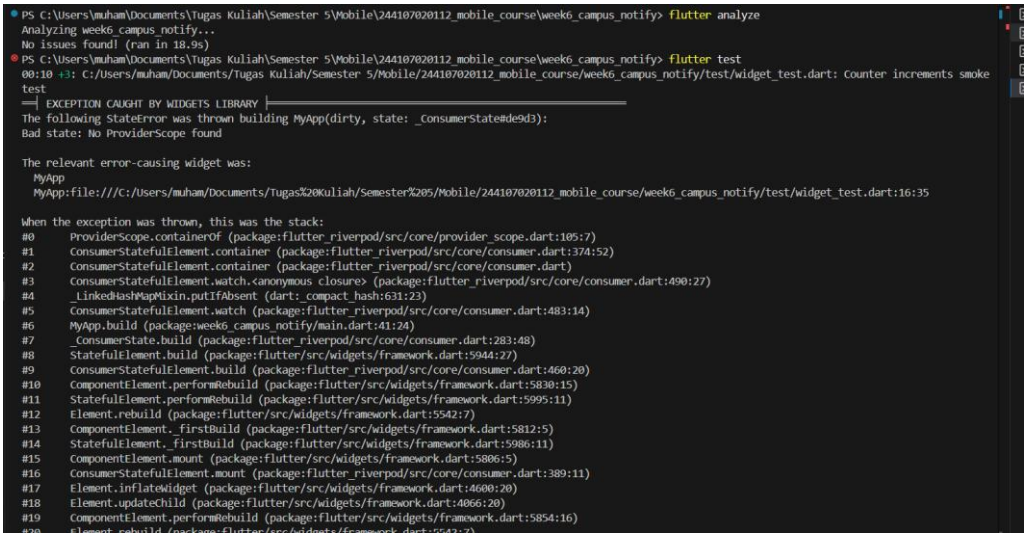
Langkah	Keterangan

1.	Background handler wajib top-level
	`@pragma('vm:entry-point')`
	`Future<void> firebaseMessagingBackgroundHandler(RemoteMessage message) async {`
	`  // Jangan akses BuildContext / Riverpod di sini.`
	`  // Tugasnya: catat / simpan ringan saja. Navigasi dilakukan saat klik.`
	`}`

2.	Tiga handler + contoh payload gabungan
	`void listenForeground(void Function(String route) go) {`
	`  _navigateCallback = go;`
	`  FirebaseMessaging.onMessage.listen((message) async {`
	`  final route = message.data['route'] ?? '/';`
	`  const androidDetails = AndroidNotificationDetails(`
	`  'pengumuman',`
	`  'Pengumuman Kampus',`
	`  importance: Importance.max,`
	`  priority: Priority.high,`
	`  );`
	`  await _local.show(`
	`  id: message.hashCode,`
	`  title: message.notification?.title ?? 'Pengumuman',`
	`  body: message.notification?.body ?? '',`
	`  notificationDetails: const NotificationDetails(android: androidDetails),`
	`  payload: route,`
	`  );`
	`  });`

3.	Matriks pengujian wajib 

Testing

Langkah	Keterangan
1.	

Refleksi

- Mengapa refresh token tidak boleh disimpan di SharedPreferences? Apa risikonya bila bocor?

  Answer :

  SharedPreferences menyimpan data dalam bentuk teks biasa (*plain text* tanpa enkripsi), sehingga sangat mudah diekstrak (misal pada HP yang di-*root* atau via *backup*). Karena refresh token memiliki masa aktif yang panjang, jika bocor penyerang dapat terus-menerus meminta *access token* baru dan membajak akun mahasiswa secara permanen tanpa memerlukan kata sandi. Wajib gunakan flutter_secure_storage (Android Keystore / iOS Keychain).

- Apa yang rusak bila onTokenRefresh diabaikan selama satu semester perkuliahan?

  Answer :

  Token FCM perangkat bisa berubah sewaktu-waktu (akibat pembaruan OS, *clear cache/data*, *reinstall*, atau rotasi otomatis Google). Jika diabaikan, backend akan menyimpan token basi (*stale token*), sehingga server menganggap pesan terkirim padahal semua notifikasi kuliah (jadwal kuliah, ujian, dll.) gagal terkirim dan tidak akan pernah masuk ke HP mahasiswa.

- Kapan memakai topik dan kapan memakai token perangkat? Beri contoh pesan kampus untuk masing-masing.

  Answer:

  - Topik: Dipakai untuk pesan broadcast publik/massal ke seluruh mahasiswa yang berlangganan tema tersebut.
    Contoh: "Perkuliahan ditiadakan besok karena Dies Natalis Kampus" (Topik: pengumuman-kampus).
  - Token Perangkat: Dipakai untuk pesan personal, rahasia, dan spesifik ke satu akun mahasiswa.
    Contoh: "Nilai KHS semester Anda telah terbit" atau *"Tagihan UKT Anda jatuh tempo dalam 2 hari"* (Pesan personal ke FCM token mahasiswa terkait).

- Bagian mana dari draf AI yang Anda tolak atau perbaiki, dan mengapa?

  Answer:

  - Penolakan penggunaan BuildContext di listener FCM: Draf AI sering menggunakan context.go() atau Navigator.of(context) di dalam callback FCM. Ini ditolak dan diganti dengan instans router langsung (router.go()), karena context belum siap (*unmounted*) saat *cold start / background* sehingga memicu crash.
  - Penambahan Izin Android 13+ & Notification Channel: Draf AI awalnya belum menyertakan POST_NOTIFICATIONS di AndroidManifest.xml serta channel prioritas tinggi, yang diperbaiki agar notifikasi tidak diblokir oleh sistem Android versi modern.